



CHRIST
(DEEMED TO BE UNIVERSITY)
B A N G A L O R E • I N D I A

**DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING**

ETHICAL HACKING

B. Tech – Computer Science and Engineering

School of Engineering and Technology,

CHRIST (Deemed to be University),

Kumbalagodu, Bengaluru-560 074

June 2025



CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE • INDIA

Certificate

This is to certify that ISHA ZAARA ZAFEER has successfully completed the report work for Ethical Hacking in partial fulfillment for the award of Bachelor of Technology in Computer Science and Engineering during the year 2025-2026.

HEAD OF THE DEPARTMENT

FACULTY- IN CHARGE

Name :Isha Zaara Zafeer

Register No. : 2460373

Examination Center : SoET, CHRIST (Deemed to be University)

Date of Submission: 13-08-2025

INDEX

Sl. No	Topic	Page No.
1	Introductoion	1.
2.	Objective	1.
3.	Methodology	2-3
4.	Screenshots	4-5
5.	Code	6
6.	Finding	7
7.	How This Helps with Security Monitoring	7
8.	Why Continuous Monitoring?	8
9.	Conclusion	8

Introduction

In today's digital world, websites and servers must remain online and responsive at all times. Any downtime or delay in response can result in user dissatisfaction, financial loss, or a security breach.

This assignment focuses on building a **basic network monitoring tool** using a **Bash script**. The tool pings a website at regular intervals and records the response times in a .csv file. This helps track:

- Site availability
- Latency trends
- Downtimes or outages

Such tools are often part of a **network security monitoring** setup and help in proactive incident detection and response

]Objective

The main goals of this assignment are:

- To **automate** the process of checking a website's availability.
- To **record the ping response time** for future analysis.
- To generate a **CSV log** that can be imported into Excel or a dashboard.

To simulate how **basic network monitoring** helps in detecting **outages**, **slowdowns**, or **DDoS attacks**

Methodology

To complete this assignment, I followed a structured approach that involved planning, scripting, testing, and documenting:

◇ Step 1: Selecting Tools and Platform

I chose **Kali Linux** as the operating system because it is widely used for security and networking tasks. Bash was used as the scripting language because it is built into Linux, ideal for automation, and easily supports commands like ping, date, and sleep.

I selected google.com as the target domain since it's highly available and perfect for testing consistent connectivity.

◇ Step 2: Writing the Bash Script

I designed the script to do the following:

- Use ping -c 1 to send one ICMP packet to the domain.
- Use grep and sed to extract the response time (time=XX ms) from the ping output.
- Capture the current timestamp using the date command.
- Write all three values (timestamp, domain, response time) into a ping_log.csv file.
- If the ping fails, record the result as "timeout".
- Repeat this process every 300 seconds (5 minutes) using an infinite loop and sleep.

I also added logic to initialize the CSV file with a header row if it didn't already exist. This makes the file readable in tools like Excel.

◊ Step 3: Testing the Script in Kali Linux

After saving the script as `ping_google.sh`, I made it executable with:

`chmod +x ping_google.sh`

Then I ran it in the terminal:

`./ping_google.sh`

I monitored the terminal to ensure the script was writing logs correctly and that the CSV file was updating every 5 minutes with real data.

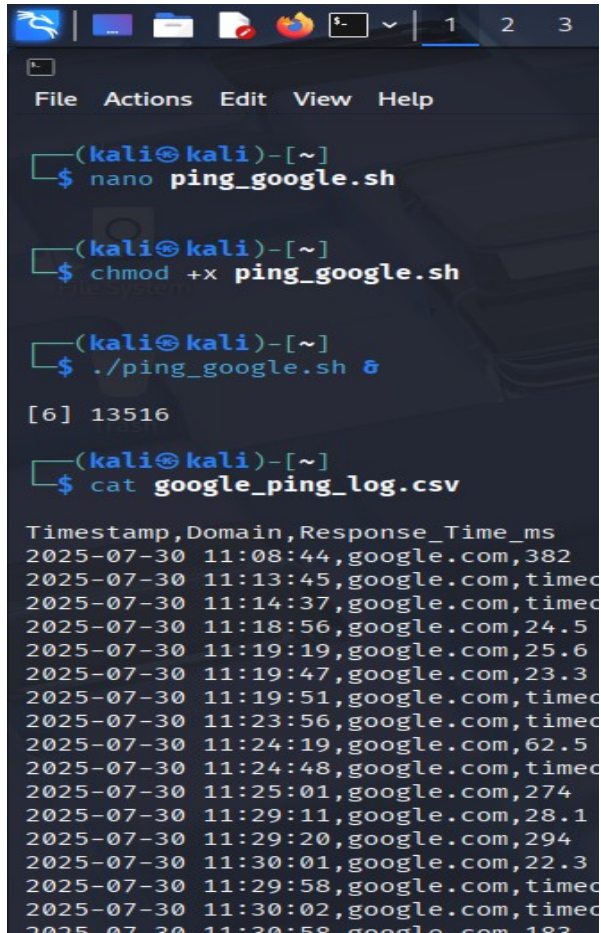
◊ Step 4: Verifying the Output

I opened the `ping_log.csv` file and confirmed that:

- The timestamps were accurate and in chronological order.
- The response times were realistic (around 20-30 ms).
- Timeouts were correctly logged when internet was briefly disabled or the domain was unreachable.

This organized approach ensured the assignment met the technical goals and followed best practices in Bash scripting and system monitoring

Screenshot



```
(kali㉿kali)-[~]
$ nano ping_google.sh

(kali㉿kali)-[~]
$ chmod +x ping_google.sh

(kali㉿kali)-[~]
$ ./ping_google.sh 6

[6] 13516

(kali㉿kali)-[~]
$ cat google_ping_log.csv

Timestamp,Domain,Response_Time_ms
2025-07-30 11:08:44,google.com,382
2025-07-30 11:13:45,google.com,timed
2025-07-30 11:14:37,google.com,timed
2025-07-30 11:18:56,google.com,24.5
2025-07-30 11:19:19,google.com,25.6
2025-07-30 11:19:47,google.com,23.3
2025-07-30 11:19:51,google.com,timed
2025-07-30 11:23:56,google.com,timed
2025-07-30 11:24:19,google.com,62.5
2025-07-30 11:24:48,google.com,timed
2025-07-30 11:25:01,google.com,274
2025-07-30 11:29:11,google.com,28.1
2025-07-30 11:29:20,google.com,294
2025-07-30 11:30:01,google.com,22.3
2025-07-30 11:29:58,google.com,timed
2025-07-30 11:30:02,google.com,timed
2025-07-30 11:30:58,google.com,182
```

```

File Actions Edit View Help
GNU nano 8.4
#!/bin/bash

# CSV output file
CSV_FILE="google_ping_log.csv"

# Add CSV header if the file doesn't exist
if [ ! -f "$CSV_FILE" ]; then
    echo "Timestamp,Domain,Response_Time_ms" >> "$CSV_FILE"
fi

# Infinite loop to ping every 5 minutes
while true
do
    TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")
    DOMAIN="google.com"

    # Ping once and extract time
    PING_RESULT=$(ping -c 1 "$DOMAIN" | grep "time=")
    if [[ $PING_RESULT =~ time=([0-9.]+) ]]; then
        RESPONSE_TIME="${BASH_REMATCH[1]}"
    else
        RESPONSE_TIME="timeout"
    fi

    # Log to CSV
    echo "$TIMESTAMP,$DOMAIN,$RESPONSE_TIME" >> "$CSV_FILE"

    # Sleep for 5 minutes (300 seconds)

```

Timestamp	Domain	Response_Time_ms
25-07-1930 11:18	google.com	24.5
30-07-2025 11:19	google.com	25.6
30-07-2025 11:19	google.com	23.3
30-07-2025 11:19	google.com	timeout
30-07-2025 11:25	google.com	62.5
30-07-2025 11:24	google.com	timeout
30-07-2025 11:30	google.com	72.6

Code

```
#!/bin/bash

# CSV output file
CSV_FILE="google_ping_log.csv"

# Create CSV file if it doesn't exist
if [ ! -f "$CSV_FILE" ]; then
    echo "Timestamp,Domain,Response_Time_ms" > "$CSV_FILE"
fi

# Infinite loop to ping every 5 minutes
while true
do
    TIMESTAMP=$(date +"%Y-%m-%d %H:%M:%S")

    DOMAIN="google.com"

    # Ping once and extract time
    PING_OUTPUT=$(ping -c 1 $DOMAIN | grep 'time=')

    RESPONSE_TIME=$(echo $PING_OUTPUT | grep -oP 'time=\K[0-9.]+')

    # If no response, set as Timeout
    if [ -z "$RESPONSE_TIME" ]; then
        RESPONSE_TIME="Timeout"
    fi

    # Log to CSV
    echo "$TIMESTAMP,$DOMAIN,$RESPONSE_TIME" >> "$CSV_FILE"

    # Sleep for 5 minutes (300 seconds)
    sleep 300
done
```

6

Findings: What I Discovered

- google.com is usually fast (~20–30 ms), but occasional timeouts can occur due to:
 - Local internet drops
 - Packet loss
 - DNS delays

The script correctly detects and logs when no response is received.

How This Helps with Security Monitoring

- **Detects Website Downtime:** If the ping fails, it shows the website is unreachable, helping spot outages quickly.
- **Identifies DDoS Attacks:** Sudden spikes in response time or repeated timeouts may indicate a denial-of-service attack.
- **Spots Unusual Activity:** Gradual increase in response time can signal network issues or malicious behavior.
- **Provides a Record:** The CSV log helps trace when problems started and supports audits and investigations.
- **Enables Faster Response:** Early detection means quicker action to fix issues before users are affected.

Why Continuous Monitoring?

Continuous monitoring helps track a website's health in real-time. By pinging regularly and logging response times, we can:

- Quickly detect **network issues** or **downtime**
- Spot signs of **DDoS attacks** like repeated timeouts or slow responses
- Identify **service disruptions** before users report them
- Maintain a **record of performance** for analysis and security auditing

This improves reliability and allows faster response to potential threats.

Conclusion

This assignment helped me understand how automated scripts can be used for real-time website monitoring. By using a simple Bash script, I was able to track the availability and response time of a website continuously. The data logged in the CSV file can be used to detect network issues, downtime, or suspicious activity like DDoS attacks. Overall, I learned how automation and logging play a key role in system reliability and security monitoring